

QA Discovery-Channel Ledger

Revision: 12 **Last modified:** 2026-08-19T17:20:00+02:00 **Status:** active
Constitution: Â§11.4.238 (automated QA must be the DISCOVERER, not the confirmer “ every defect found outside the automated HelixQA regime is itself a coverage-escape release blocker, not merely a bug to fix).

Purpose

Every defect this project closes MUST record **which channel found it:** automated-helixqa (the standing HelixQA regime caught it) or an **out-of-band channel** “ manual-qa, operator-report, agent-code-reading, or incidental-discovery. Per Â§11.4.238(C), every out-of-band entry MUST carry a **coverage-escape audit:** why the automated regime missed it (cited to the specific missing/blind check, never “œwe didnâ€™t think of itâ€”), and the new or strengthened automated check “ with its own Â§11.4.115 RED capturing the escaped defect “ that closes the gap.

Per Â§11.4.238(E), this project tracks the discovery-channel split over time and drives the out-of-band share toward zero. **Honest boundary (Â§11.4.6):** this ledger starts wherever this projectâ€™s actual history is “ see the retroactive seed entries below, all found via agent-code-reading during the 2026-08-07/2026-08-08 governance audits, exactly the channel this anchor targets. Full retroactive audit of every historical closure in docs/Fixed.md (~62 items) is NOT attempted here “ that is out of proportion for a ledger just being stood up, and would itself be a Â§11.4.6 fabrication if backfilled without real investigation per entry. The ledger is honest about starting now, not claiming a false complete history.

Schema (per entry)

Field	Meaning
id	The workable item id (BOB-NNN) if one exists, else a short slug
date	ISO date the defect was found
channel	automated-helixqa manual-qa operator-report agent-code-reading incidental-discovery
summary	One line: what was wrong
escape-audit	(out-of-band only) which check was missing/blind, cited by name/path
new-check	(out-of-band only) the new/strengthened check that now catches this class, with its RED-capture evidence

Entries

RD2-22 “PATCH /schedules/{id} and PUT /theme unauthenticated with a token set

- **id:** RD2-22 (governance-audit item, not yet a BOB-NNN)
- **date:** 2026-08-07 (found by the 2026-08-07 governance audit), confirmed still open + fixed 2026-08-08
- **channel:** agent-code-reading “ found by direct source inspection (comparing sibling routes’™ Depends(require_api_token) presence), not by any test or HelixQA run.
- **escape-audit:** tests/security/test_hooks_schedules_auth.py’™s _MUTATING enumeration (the ONLY automated check covering this auth surface) was scoped to exactly 4 routes (POST/DELETE hooks, POST/DELETE schedules) when the original RW-02 fix landed “ it was never extended when PATCH /schedules/{id} and PUT /theme were added as separate routes. The check existed, ran green, and was simply never told about two of the six routes it should have covered “ a scope gap, not a broken check (per the ledger’™s channel split, this is the “œgenuinely uncovered” class, not “œexisted-but-missed”).
- **new-check:** _MUTATING extended to all 6 routes (tests/security/test_hooks_schedules_auth.py), RED-captured (confirmed 4/4 new-route assertions failed with expected 401, got 200 before the fix), now GREEN (31/31). See docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-22 for the full before/after evidence.

RD2-41a “ scripts/docs_chain.sh Step 1/3 silently no-op” d on every run

- **id:** RD2-41 (governance-audit item)
- **date:** 2026-08-08
- **channel:** agent-code-reading “ found by directly invoking docs_chain.sh and reading its own printed error line, then tracing the hardcoded path in source.
- **escape-audit:** **no automated check existed at all** for “œdoes docs_chain.sh’™s Step 1 actually run the real DB export” “ the script printed an ERROR: line on every failure but nothing consumed/ asserted on that exit status anywhere in the test suite or pre-build gate; a human had to actually run it and read the output. Genuinely uncovered, not a broken check.
- **new-check:** tests/unit/test_docs_chain_binary_resolution.sh “ real-invocation test asserting Step 1 resolves a real binary and reaches the real validate call. RED-captured (confirmed failing against the pre-fix script: FAIL: Step 1/3 still reports 'binary not found'), now GREEN.

RD2-41b “ scripts/pre_build_verification.sh invariant 17 silently SKIPPED on every run

- **id:** RD2-41 (governance-audit item, same root cause as RD2-41a)
- **date:** 2026-08-08
- **channel:** agent-code-reading “ found while fixing RD2-41a, by checking for the identical bug pattern in this sibling script per the project’s own extend-to-all-cases discipline.
- **escape-audit:** this is the sharpest instance in this ledger of “the checker itself was blind”: invariant 17 IS the pre-build gate’s own workable-items DB-integrity check, and it was silently SKIPPING (not failing “ the script’s own else branch treats a missing binary as an honest skip, not a failure) on every single pre-build run since the path was wrong. A gate whose own precondition-check is broken cannot report the truth about the thing it gates “ an existed-but-missed class defect (the check existed, ran, and was blind), the more dangerous of the two classes per this ledger’s schema.
- **new-check:** tests/unit/test_pre_build_workable_items_invariant.sh “ real-invocation test asserting invariant 17 reaches a real PASS/FAIL verdict instead of silently skipping. RED-captured (confirmed failing against the pre-fix script via git stash/restore), now GREEN. **Residual, still-open finding from this same investigation** (tracked as RD2-41’s own note in docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md, not yet closed): a SEPARATE, earlier mutation-marker carrier false-positive in the same script aborts the ENTIRE pre-build gate before invariant 17 is ever reached in a full run “ meaning this fix, while correct, is not yet provably exercised end-to-end in production; the ledger records this honestly rather than claiming full closure.

BOB-008 “ DB’s ĨZMD body drift + missing enumerated unblock choices

- **id:** BOB-008
- **date:** 2026-08-08
- **channel:** agent-code-reading “ found by running workable-items diff/validate during a routine sync check, not by any standing automated gate (no CI job runs workable-items diff on a schedule).
- **escape-audit:** no automated check runs workable-items diff/validate as a standing, scheduled, or write-seam-triggered gate “ Â§11.4.106(F)’s commit-seam sync hook (which would have caught the DB write in commit 54e313f landing with no matching docs/Issues.md update) does not exist in this repo. Genuinely uncovered.
- **new-check: closed 2026-08-08.** scripts/pre_build_verification.sh invariant 17 extended to run workable-items diff (DB-vs-Markdown divergence) alongside validate (internal DB invariants) “ it now catches BOTH the Â§11.4.148(D3) enumerated-choices class AND the DB’s ĨZMD body-drift class. RED-captured via tests/unit/test_pre_build_workable_items_diff_check.sh’s Â§1.1 paired mutation (a deliberately desynced docs/Issues.md “ confirmed FAIL

with the diff check absent, confirmed correct divergence-specific FAIL, distinguished from the separate pre-existing BOB-009/010 validate issue so the test does not tautologically reuse an unrelated failure “ then GREEN after the fix, with a companion assertion that the real synced tree reports no false divergence).

- **2026-08-18 update (BOB-073 / RD2-04, recurrence, honest residual finding):** despite the 2026-08-08 invariant-17 diff extension above, `workable-items diff --db docs/workable_items.db --issues docs/Issues.md --fixed docs/Fixed.md` found **9 separate DBâ†”i, ŹMarkdown drifts** (not just the original BOB-008 one) when an agent manually re-ran it during the BOB-072/BOB-073 fix session (commit 82d9842) “ evidence in `docs/qa/BOB-072-073/`. **Channel for this recurrence: agent-code-reading** (an agent ran the tool by hand while fixing an unrelated sibling item, not a standing scheduled/blocking gate). **Escape-audit:** the invariant-17 diff check landing in source is necessary but not proven sufficient “ this ledger does NOT have evidence that invariant 17 runs as a hard, blocking write-seam gate on every commit that touches `docs/workable_items.db` (per Â§11.4.106(F)â€™s commit-seam requirement); 9 drifts accumulating between 2026-08-08 and 2026-08-18 is consistent with either (a) the check existing but only being invoked when an agent remembers to run `scripts/pre_build_verification.sh` by hand, or (b) some DB writes in that window landing via a path invariant 17 does not intercept. **Recorded honestly as UNKNOWN which mechanism** (Â§11.4.6) rather than guessed. All 9 reconciled + closed this session (see commit 82d9842 for the per-item DB-vs-MD authoritative-side decisions); diff now reports 0. **New check (still owed, not yet authored):** a real commit-time (pre-commit or commit-push-all.sh stage per Â§11.4.234) hook that runs `workable-items diff` and BLOCKS the commit on any divergence touching `docs/workable_items.db`, `docs/Issues.md`, or `docs/Fixed.md` “ turning invariant 17 from a pre-build-time report into a write-seam gate.

RD2-42 “ podman ps reports a container “œUp “ (healthy)â€” while its process is genuinely dead

- **id:** RD2-42 (governance-audit item)
- **date:** 2026-08-09
- **channel:** incidental-discovery “ surfaced while attempting the RD2-22 live curl-verify: `curl http://localhost:7187/health` returned HTTP=000 (connection refused), while `podman ps` simultaneously reported `qbittorrent-proxy: Up 15 hours (healthy).podman exec qbittorrent-proxy ... returned OCI runtime error: crun: ... is not running and podman inspect ... .NetworkSettings.Ports was empty {}` “ the containerâ€™s actual crun process was dead (no restart count, no OOMKilled flag recorded “ podmanâ€™s own state was simply stale/desynced from reality), most likely fallout from the same host session-kill mechanism already tracked in `docs/incidents/` reaching into the rootless-podman container process tree.

- **escape-audit: no automated check exists at all** for “podman ps”’s reported state consistent with the container’s real crun/process state” this is exactly the 11.4.196(F)/11.4.201(6) “configured % in use” false-null class applied to container orchestration: a healthy-looking podman ps line is not proof the service is reachable. Genuinely uncovered” no pre-build/pre-test gate probes the live stack’s actual reachability before a test run assumes it.
- **new-check:** not yet authored (tracked, not yet closed). Candidate: a tests/fixtures/services.py-level or pre-build-gate-level real-reachability probe (curl/socket-connect to each mapped port, not podman ps text) run before any live-stack- dependent test session, with an honest recreate-and-retry path” mirroring what this session did manually (./start.sh --recreate, confirmed HTTP=200 after ~90s).
- **resolution applied this session:** ./start.sh --recreate (the project’s sanctioned orchestrator, never raw podman restart) “confirmed curl :7187/health â†’ 200 after the stack’s normal ~90s startup window.

RD2-43 “bare python3 -m pytest silently fails ALL collection via a stale ~/.local rpds build

- **id:** RD2-43 (governance-audit item)
- **date:** 2026-08-09
- **channel:** incidental-discovery “surfaced when relaunching the interrupted `tests/contract/`
 - tests/unit/regression sweep with a barepython3 -m pytest invocation: immediate ModuleNotFoundError: No module named “rpds.rpds” at collection time (inside the schema thesis` pytest-plugin’s import chain), aborting the ENTIRE sweep before a single test ran.
- **escape-audit:** host-level Python version drift “the system python3 resolves to Python 3.14.6, but ~/.local/lib/python3/site-packages/rpds/rpds.cpython-313-x86_64-linux-gnu.so is a native extension built for the 3.13 ABI (real ABI mismatch, confirmed via ls .../rpds/*.so + python3 --version). The project’s own .venv (Python 3.14.6, .venv/bin/ python3 -c "import rpds" succeeds) is the correct, working interpreter “no automated check enforces “tests are always run via .venv/bin/python3, never a bare python3” anywhere in this repo’s own tooling (no Makefile/wrapper script that fails closed on the wrong interpreter). Genuinely uncovered; this is an environment-fragility class distinct from the source-level bugs this ledger otherwise tracks, but it silently produces a 100%-collection-failure that could be misread as “every test in the suite is broken” by anyone (human or agent) who doesn’t already know to check the interpreter.
- **new-check:** not yet authored (tracked, not yet closed). Candidate: a thin scripts/run_tests.sh wrapper (or a pyproject.toml/CI-adjacent guard) that verifies sys.prefix resolves inside .venv/ before invoking pytest, failing closed with an actionable message rather than a confusing plugin-internals traceback.

- **resolution applied this session:** relaunched the sweep via `.venv/bin/python3 -m pytest` confirmed genuinely running (real per-test PASS lines, not a collection error).

RD2-44 “test_get_existing_search_returns_200”s 120s poll window had near-zero real margin

- **id:** RD2-44 (governance-audit item, same root-cause FAMILY as the already-fixed `tests/e2e/test_full_pipeline.py` timeouts, but a genuinely separate finding/fix “that fix did not touch this file)
- **date:** 2026-08-09
- **channel:** incidental-discovery “surfaced re-running `tests/integration/test_merge_api.py` (RD2-26a’s own deliverable) against the actually-live stack for the first time since its original mocked-service replacement; it had only ever been run with the stack unreachable (28 passed / 20 skipped) until this session.
- **escape-audit:** RD2-26a’s own author (a parallel subagent, same session) had no way to discover this “the file was authored and its author’s own live-verification attempt happened while the stack was down, so it never actually exercised this code path against a live, contended host before committing. No automated check re-runs the live-service test suite on a schedule independent of whether an agent happens to have the stack up at authoring time “genuinely uncovered.
- **new-check:** the test itself IS the check; it was simply mistimed. Live-measured real search completion (42 trackers, real network calls, under real concurrent 4-subagent host load): ~118s once, ~298s under heavier concurrent load “both within a few seconds of, or past, the old 120s ceiling. Widened to 300s in `tests/integration/test_merge_api.py`, matching the `test_full_pipeline.py` pattern’s order of magnitude. RED-adjacent evidence: the original 120s-deadline run genuinely failed (`assert 'running' != 'running'`) against the live stack; post-fix re-run: 1 passed in 297.72s.

INC-2026-08-10 “GCM auth-bypass mutation left in qBitTorrent-go/internal/db/crypto.go during Agent H’s RED window

- **id:** INC-2026-08-10 (this-session incident; not a governance-audit item “inline coverage-escape captured during autonomous work by the conductor)
- **date:** 2026-08-10
- **channel:** `automated_background_scan` “a background security-review plugin (out-of-cycle from the pre-build regime) caught the mutation during Agent H’s Case (c) RED-then-GREEN window (~few seconds) while it was authoring RD2-31 chaos tests in `qBitTorrent-go/tests/integration/jackett_db_test.go`.
- **escape-audit:** subagent briefing template for RD2-31 said “œDo NOT touch production code paths “ONLY the test file” but was PROSE, not mechanically enforced. Agent H interpreted narrowly and mutated `qBitTorrent-go/internal/db/crypto.go` `Decrypt` (`if err !`

= nil â†’ if false && err != nil, swallowing GCM authentication failures) for its RED assertion, then restored to byte-identical HEAD. The window was small but real â€” a Â§11.4.84 working-tree-quiescence violation. No boba-side pre-tool-use hook would have caught the mutation during the window (only a post-fact scan would); the out-of-cycle security scan caught it BEFORE restoration. Genuinely uncovered by the in-cycle regime.

- **new-check:** Â§11.4.84 fence LANDED â€”
 - **Layer 1** (subagent briefing template with three mandatory isolation patterns: git worktree per Â§11.4.179, atomic mutate-run-restore Bash tool call with trap, mutation-testing tool operating on isolated copy) APPLIED to every subsequent subagent dispatch (proven: Agents K + L both used the strengthened briefing without violation).
 - **Layer 3** (scripts/pre_build_verification.sh invariant 23 CM-NO-PRODUCTION-MUTATION-RESIDUE, with FIXTURE_ROOT env-testable golden-good/golden-bad fixtures under scratchpad/agent-L-fixtures/) LANDED â€” real-repo PASS today (0 residue in production paths).
 - **Layer 2** (post-tool-use hook enforcing mid-window detection) DEFERRED pending Claude Code runtime-capability verification (currently PreToolUse hooks only per Â§11.4.109 precedent).
 - Design doc: scratchpad/task-20-84-fence-design.md.
- **resolution applied this session:** Agent Hâ€™s crypto.go restoration was clean (verified via git status empty + git diff empty + full-tree grep for MUTâ€™ATED patterns in production paths = empty); no commit contamination (main stream was holding for atomic commit per Â§11.4.121). Â§11.4.84 fence Layer 1 + Layer 3 both landed and verified. Layer 2 tracked as Â§11.4.197 follow-up.

TMUX-OOMD-2026-08-12 â€” tmux sessions SIGKILLd by systemd-oomd under user-slice pressure, despite MemoryMax=infinity

- **id:** TMUX-OOMD-2026-08-12 (not yet a BOB-NNN â€” cross-project escape traced through the upstream vasic-digital/tmux fix TMX-083, v1.0.42)
- **date:** 2026-08-12
- **channel:** operator-report â€” the operator noticed sessions dying â€œas soon as we continue work with this projectâ€œ. No automated gate in this project or its upstream had exercised this failure class.
- **escape-audit:** neither the tmx projectâ€™s own test suite (which covers cgroup Max= / TasksMax= / CPUQuota properties but not ManagedOOMPreference) nor the boba projectâ€™s HelixQA + Challenges regime (host-safety mandates cover CONST-033 host power classes but do not probe systemd-oomdâ€™s effect on tmux scopes) had a check for â€œdoes the tmux scope survive a user-slice memory-pressure spike under systemd-oomd?â€œ. The Â§11.4.201(6) FALSE-NULL class applied to the test surface: the suite scanned the wrong axis and returned a confident zero on a real defect. Genuinely uncovered â€” no automated check existed on either side. Predecessor

sighting RD2-42 (2026-08-09, incidental-discovery) had already noted the same host session-kill mechanism reaching into rootless-podman container process trees; this operator report is the tmux-facing symptom of the same killer.

- **new-check:** challenges/scripts/tmux_survives_oom_pressure_challenge.sh (boba side; auto-wired into run_all_challenges.sh)
 - upstream scripts/tests/59_oom_preference_avoid.sh (tmx side). Both are Â§11.4.115 RED-capable via RED_MODE=1. Live-verified on the operatorâ€™s host 2026-08-12: pre-fix RED_MODE=1 PASS (ManagedOOMPreference=none â€” defect reproduced), post-fix RED_MODE=0 PASS (ManagedOOMPreference=avoid â€” regression guard confirmed). Perfect Â§11.4.115 polarity flip. Full audit at docs/qa/coverage-escape-tmux-oomd-20260812/audit.md. Root cause fixed upstream in vasic-digital/tmux v1.0.42 (TMX-083, commits 6f9eae8 + merge 92ef3a0), pushed ff-only to both remotes per Â§11.4.113.

RD2-00 / BOB-068 â€” unattributed, unreviewed â€œAuto-commitâ€” mechanism pushing to main mid-work

- **id:** BOB-068 (RD2-00) â€” status Queued, still OPEN (root cause narrowed, not eliminated)
- **date:** first observed 2026-08-08; re-confirmed present in git history as of 2026-08-18 (20 bare Auto-commit commits total across this repoâ€™s history â€” confirmed via `git log --oneline --all --grep="^Auto-commit$" | wc -l`, e.g.Â 54e313f, 9c8f684, 743097a, de9270b, 1c36777, 41179c2, 7c529ca).
- **channel:** agent-code-reading â€” found by re-running `git log` mid-investigation during the 2026-08-08 governance audit and discovering two NEW Auto-commit commits (9c8f684, 743097a) had landed on main **while the audit was already in progress**.
- **escape-audit:** no automated check exists anywhere in this repo for â€œdoes a commit reaching main carry an ATM-NNN citation / TDD trail, or is it a bare/templated message from an unattributed sourceâ€”â€” Â§11.4.84 working-tree-quiescence has no mechanical guard on this specific path (writes arriving via an ordinary `git pull --ff` from a second session/host with push access to the same remotes, per the RD2-00 root-cause pass in docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md: `git reflog` proved the two newest commits were never authored on the investigating host â€” they fast-forwarded in via `pull`, and their commit timezone (+0500) does not match the investigating hostâ€™s own (+0300 MSK)). Genuinely uncovered â€” no gate flags an unattributed/unticketed commit reaching main.
- **new-check:** not yet authored (tracked, not yet closed â€” BOB-068 remains Queued). Followup filed: **BOB-106** â€” a Â§11.4.84 quiescence-check helper scanning the commit range since the last known-good release tag and FAILing on any bare/templated commit message (^Auto-commit\$, ^sync:, etc.) with no ATM-NNN reference, wired into scripts/pre_build_verification.sh or the Â§11.4.234 commit-push-all.sh endpoint.

- **resolution status:** root cause NARROWED, not eliminated â€” docs/GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-00 Update 2 confirms the mechanism is (most likely) the operatorâ€™s own second session/device bypassing per-commit TDD/review discipline on ITS side, not an unidentified external actor; this downgrades severity from P0 to P1 but does not close the item.

BOB-072 â€” docs/workable_items.db machine-caught SSoT integrity violations (90% of closures had zero audit trail)

- **id:** BOB-072 (RD2-03) â€” **closed 2026-08-18**, commit 82d9842
- **date:** found 2026-08-08 (governance audit); closed 2026-08-18
- **channel:** agent-code-reading at discovery (2026-08-08, an agent manually ran constitution/scripts/workable-items/bin/workable-items validate --db docs/workable_items.db during the audit â€” exit 1, 2 real violations: BOB-009/BOB-010 both had a closure evidence_path that does not resolve). Beyond the toolâ€™s own catch, a direct SQL sweep at the time found 56 of 62 closed items (90%) with zero item_history rows.
- **escape-audit:** at the time of the 2026-08-08 discovery, invariant 17 of scripts/pre_build_verification.sh â€” the ONLY standing gate wired to run workable-items validate â€” was ITSELF silently SKIPPING on every run (the sibling coverage escape already ledgered above as **RD2-41b**, same date, same audit): a gate whose own precondition-check is broken cannot report the truth about the thing it gates. RD2-03â€™s violations therefore went unnoticed by the standing regime until RD2-41bâ€™s own fix (same session) restored invariant 17 to a real PASS/FAIL verdict â€” the existed-but-missed class (Â§11.4.201(6) FALSE-NUL), not genuinely-uncovered.
- **new-check:** challenges/scripts/workable_items_integrity_challenge.sh extended with Â§11.4.115 RED_MODE polarity (RED_MODE=1 reproduces on the pre-fix DB snapshot in docs/qa/BOB-072-073/, RED_MODE=0 default GREEN guard â€” both validate and diff exit 0), 3/3 PASS live-verified this session. Combined with the already-landed RD2-41b fix, invariant 17 is now the mechanized regime for this defect class.
- **Note (discovery-method signal, not a channel reclassification):** this sessionâ€™s closing verification (commit 82d9842) captured its before/after evidence via the ACTUAL tool (workable-items validate/diff exit codes), not raw SQL or narrative inspection â€” the correct, mechanizable shape Â§11.4.238 wants going forward, even though the ORIGINAL 2026-08-08 discovery was still an agent running that tool by hand during an ad-hoc audit, not a standing/scheduled gate firing on its own. The channel is honestly recorded as agent-code-reading, not automated-helixqa â€” the tool was correct, its invocation was not yet automatic.

META-11.4.227B-2026-08-18 â€” Â§11.4.227(B) anchor-block-integrity verified BY HAND, not by a mechanical gate

- **id:** none yet (governance meta-finding, not a BOB-NNN â€” the underlying anchor corpus is constitution-submodule-owned; this ledger records the escape because it was found during boba-repo work this session)
- **date:** 2026-08-18 (this sessionâ€™s Â§11.4.140/Â§11.4.141 anchor-number-collision resolution, Phase 0 â€” boba commit 136a22c, constitution commits 5ed8c80..e5f2891)
- **channel:** agent-code-reading â€” the Phase 0 subagent manually computed md5 hashes of the moved anchor bodies and manually grepped ### Â§11.4.140 / ### Â§11.4.141 occurrence counts, pasting the results verbatim into the commit message (git show --stat 136a22c) to prove Â§11.4.227(B) block-integrity (exactly-once per anchor, byte-identical lockstep across mirrors).
- **escape-audit:** Â§11.4.227(B) is fully specified in constitution prose (â€œpropagation gates count BLOCK-STARTS never bare literals: exactly-once per anchor per fileâ€; lockstep content-hash equality across the mirror setâ€; anchor-number collisions FAILâ€) but its own gate (CM-ANCHOR-BLOCK-INTEGRITY) is explicitly documented, by the anchorâ€™s own text, as â€œgate-code = separate work item, NOT claimed shipped.â€ The Â§11.4.140/Â§11.4.141 collision that Â§11.4.227 itself cites as its founding forensic example was STILL only caught/fixed by a human-in-the-loop agent running ad-hoc md5sum/grep commands, not by a runnable script â€” the textbook Â§11.4.227 finding (413 named CM-* gates, 58% unimplemented) recurring live, in this repo, this session.
- **new-check:** OUT OF SCOPE for this ticket (constitution-submodule-owned mechanism; per this taskâ€™s own constraints, boba may not touch anything constitution-related). Followup filed: **BOB-105** â€” a boba-side, read-only challenge that greps every governance file this projectâ€™s constitution submodule checkout exposes for ### Â§11.4.NNN heading occurrences and FAILs on >1-per-anchor-per-file or on two DIFFERENT anchor bodies sharing one NNN, implementing (from the consumer side, read-only) the check CM-ANCHOR-BLOCK-INTEGRITY names but does not yet run.

SCRATCH-LOSS-2026-08-18 â€” Phase 1a subagentâ€™s declared source-material inputs absent at dispatch time

- **id:** none yet (this-session incident, constitution-curriculum work stream, not a BOB-NNN)
- **date:** 2026-08-18
- **channel:** agent-code-reading â€” the Phase 1a subagent (a1cc331d) discovered, at task start, that 5 source files its own task brief named as required reads (curriculum_amendment_plan_v1.md, ai_curriculum_modules_27_35_extracted.md, and three curriculum_analysis_modules_*.md gap-analysis files) were NOT present in the session scratchpad. Self-documented in .superpowers/sdd/task-phases1a-report.md:21. Independently re-verified during

this BOB-069 audit: curriculum_amendment_plan_v1.md is STILL absent from the live scratchpad directory as of this writing, while the other four listed inputs now exist (apparently re-created by a later pass).

- **escape-audit:** root cause per the Phase 1a subagent's own investigation: "the prior stub-expansion subagent (ae59171f) apparently hit its session rate limit before writing them" a "§11.4.147(e)-class API-quota-exhaustion crash landing mid-deliverable, with no mechanical dependency check verifying a declared upstream artifact actually exists before a downstream subagent is dispatched to consume it. No automated check exists in this project's orchestration tooling that a task brief's named "read this first" inputs are present at dispatch time" genuinely uncovered; discovered only because the downstream subagent noticed and self-reported rather than silently fabricating content against an absent source (which its own brief's no-fabrication instruction " [MATERIAL-THIN] marking " correctly steered it away from).
- **new-check:** not yet authored. Followup filed: **BOB-107** " a pre-dispatch precondition check in the orchestration layer verifying every file a task brief cites as a required input exists and is non-empty before the downstream agent is spawned, failing closed with an actionable "missing input, respawn the producer" message.
- **Honest boundary (§11.4.6), stated explicitly:** the task brief that dispatched this BOB-069 audit described this finding as "scratchpad reset between sessions" that specific mechanism (an OS-level / tmp wipe or session-boundary reset) is NOT what the cited evidence actually shows; the evidence instead shows a crashed/rate-limited PRODUCER subagent that never wrote its declared deliverable, discovered by an honest downstream subagent. This entry records the mechanism the evidence actually supports, not the dispatching brief's hypothesis, per this ledger's own anti-fabrication discipline (§11.4.6/§11.4.238's "no fabricated entries" instruction).

CODEGRAPH-1.5.0-GITIGNORE-2026-08-18 "

CodeGraph 1.5.0 re-index walked into nested-.gitignore-excluded node_modules trees (63x file-count blowup)

- **id:** none yet (this-session incident; followup tracked as BOB-104)
- **date:** 2026-08-18 " discovered by BOB-075 subagent a70f216f (per .superpowers/sdd/progress.md:78, dispatched as Task #47) while attempting a mechanical docs/codegraph/Status.md regen; landed in commit e6162f7 (fix(docs,BOB-075): refresh docs/features/Status.md + docs/codegraph/Status.md staleness).
- **channel:** agent-code-reading " the subagent ran codegraph init . --force for real on this host (the doc's own documented Option-A mechanical-regen mechanism), observed the run walk **32,260 files / 514,456 nodes / 724,013 edges** (1.8 GB DB, still growing) before deliberately aborting it " a ~63x — **blowup** versus the

2026-06-06 baseline of 509 files / 8,906 nodes for the same repository shape “ then root-caused it by direct inspection, not by any standing test or HelixQA run. Full evidence + before/after headers: docs/codegraph/Status.md Revision 2 (lines 87-124) and docs/qa/BOB-075/{before_state,after_state}.txt.

- **escape-audit:** root cause CONFIRMED (not guessed, §11.4.6) via git check-ignore -v: nested frontend/.gitignore:10 (/node_modules) and extension/.gitignore:2 (node_modules/) both correctly exclude their respective node_modules/ trees for git itself (git ls-files frontend/node_modules extension/node_modules ‘ 0 rows both), yet codegraph init on CodeGraph **1.5.0** walked into both trees anyway (365 MB + 236 MB combined) “ a CodeGraph 1.5.0 regression in honoring **nested** (non-root) .gitignore files, not honoring the same exclusion path git itself respects. frontend/ and extension/ were added to this project after the 2026-06-06 CodeGraph setup and had never been exercised against this exclusion path before “ **genuinely uncovered:** no automated check exists anywhere in this repo that re-indexing honors nested .gitignore scope, and the tool’s own documented “zero-config, exclusion driven by .gitignore” claim (the 2026-06-06 Status.md entry) was never re-verified after frontend// extension/ were added or after the tool moved from the documented 0.9.9 to the actually- installed 1.5.0.
- **new-check:** not yet authored. Followup filed: **BOB-104** “ author a challenge (challenges/scripts/codegraph_gitignore_honor_challenge.sh or equivalent) with §11.4.115 RED_MODE polarity: RED_MODE=1 reproduces the blowup against the live nested-.gitignore tree, RED_MODE=0 asserts a resync stays within the documented baseline order of magnitude. Corrective action for the underlying tool defect itself: file upstream at github.com/vasic-digital/codegraph (tracked as this ledger entry’s new-check follow-through, same scope as BOB-104).
- **Honest boundary (§11.4.6):** the run was aborted (SIGKILL) per §12.6/§12.11 host-resource discipline before it could complete or corrupt state “ no partial index was left claiming to be valid (codegraph status confirmed “index truncated” post-kill, and the truncated .codegraph/codegraph.db* is gitignored, never tracked). This project’s CodeGraph index is therefore **NOT live** as of this entry; the last VALIDATED sync remains the 2026-06-06 baseline (7 PASS / 0 FAIL, pre-frontend/extension tree shape).

FORCED-LOGOUT-2026-08-18-2ND “ user@1000.service SIGKILLed at 20:50:59, cascade kill of user session, operator perceived as forced logout

- **id:** BOB-116 (this-session incident; corrected 2026-08-18 from an informal “BOB-076” session-commit label that collided with a distinct, pre-existing, unrelated DB item “ BOB-076 is Type=Task, the RD2-09 jackett fork bump, minted 2026-08-15 by commit 99a486e “ per §11.4.54 no-ID-reuse; see the NOTE at the top of docs/incidents/2026-08-18-perceived-forced-logout-2nd.md and closure commit 2861920. followups tracked as Task #77 [wire

- challenge] / Task #78 [host_no_auto_poweroff_challenge perf regression] / Task #79 [SIGKILL-source attribution long-run]).
- **date:** 2026-08-18 " the 2nd forced-logout on this project (1st was 2026-07-07, produced Â§12.12 anchor); operator sent a CRITICAL mid-turn message reporting they had been "œfully logged out from host milosvasic account" after returning from lid-closed state to find themselves at the GDM greeter.
 - **channel:** operator-report " the incident was discovered by the operator observing the physical state (logged out at the GDM greeter after lid re-open at 20:51:06). No standing check monitored the resource-pressure signatures that preceded it (Â§12.12 EAGAIN cascade at 20:45:48, 15 GB pathological ugrep from Task #52 subagent, HTTP flood at 20:49:00, multi-fleet concurrent container pressure). systemd itself logged the user@1000.service: Main process exited, code=killed, status=9/KILL but no gate consulted that log to escalate before session termination. Full evidence: docs/qa/BOB-116/{journalctl_20-40_to_20-52.log, oomctl_snapshot.log, psi_readings.log, ps_LRSS_snapshot.log, challenge_pass.log, challenge_polarity_forced_fail.log, const033_challenge_pass.log, lid_and_session_events.log} (initially referenced as BOB-076 informal label, corrected 2026-08-18) and docs/incidents/2026-08-18-perceived-forced-logout-2nd.md.
 - **escape-audit: root cause CONFIRMED as UNCONFIRMED** (Â§11.4.6 honest " the exact mechanism that delivered SIGKILL to user@1000.service is not attributable from the systemd journal; kernel OOM never fired, systemd-oomd never triggered, CONST-033 forbidden mechanisms never invoked, HandleLidSwitch was ignore).
Contributing factors CONFIRMED: (i) Â§12.12 thread-exhaustion EAGAIN cascade at 20:45:48 " jackett SocketException (11) Resource temporarily unavailable to iptorrents+kinozal+rutracker simultaneously, exactly 5 minutes before the SIGKILL; (ii) 15 GB pathological ugrep from Task #52 subagent (ugrep -o with .\{0,120\} variable-length context + 3-way alternation against 14K-line CLAUDE.md) " reaped post-relogin, freed 16 GB immediately (PSI Avg10 1.77 â†’ 0.08); (iii) multi-fleet concurrent pressure (boba stack + sibling "œshlomi" claude session"s helix-* stack + lava-postgres + 6+ MCP servers + Yandex + JetBrains + ollama on one user.slice). NO automated check existed anywhere in this repo that (a) monitored user@1000.service health, (b) detected the leading Â§12.12 EAGAIN-cascade signature before the crisis window, (c) capped subagent grep -o variable-length-context patterns against multi-MB files. Also FOUND during triage: no_suspend_calls_challenge.sh was pre-existing FAIL from false-positive on scratchpad/ + .superpowers/sdd/ files that CARRIER the rule text (Â§11.4.201(1) false-positive-refusal class) " a separate coverage escape of its own, fixed same commit.
 - **new-check:** challenges/scripts/resource_pressure_signature_challenge.sh (NEW, committed as 1f42357) " 5-signature proactive detector: SIG-1 process >5 GB RSS (forensic FACT: 15 GB ugrep), SIG-2 thread util >70% of ulimit -u (Â§12.12 crisis: 95%), SIG-3 EAGAIN cascade in container logs last 15min (this incident: 4 trackers @ 20:45), SIG-4 user.slice PSI full

avg60 >50 (half the systemd-oomd 90% trigger), SIG-5 pathological-regex cmdline (ugrep -o + .\{N,M\} + \| alternation) â€” PASS on clean state, FAIL when thresholds crossed. Live evidence in docs/qa/BOB-116/challenge_{pass,polarity_forced_fail}.log (initially referenced as BOB-076 informal label, corrected 2026-08-18). Not yet wired into pre_build_verification.sh nor a systemd-user timer â€” **Task #77** owns that wiring (challenge exists as a runnable artifact, not a scheduled invariant, until #77 lands). Also **corrective action for CONST-033 challenge false-positive**: scripts/host-power-management/check-no-suspend-calls.sh extended EXCLUDE_PATHS to skip scratchpad/ + .superpowers/sdd/ â€” challenge returns to PASS.

- **Correction (Â§11.4.209 independent review, .superpowers/sdd/task-review-457cca4-a7e55f9-report.md, IMPORTANT-1; remedied this-session, task #78)**: the original â€œÂ§11.4.115 polarity verifiedâ€” claim above was a Â§11.4/Â§11.4.6 metric-layer bluff â€” the only forced-fail evidence at the time (challenge_polarity_forced_fail.log) ran with SIG1_MAX_PROC_RSS_GB=0, a THRESHOLD mutation that trips on every process, proving the comparison operator works but not that the detector catches the ACTUAL pathological state; SIG-2/SIG-3/SIG-4/SIG-5 had no polarity evidence at all. Fixed forward: five REAL per-signature RED fixtures now live under challenges/fixtures/resource_pressure/ â€” a genuine >5.5 GB page-resident process (SIG-1), a genuine >70%-of-(subshell-lowered)-ulimit thread-utilization ratio measured against the REAL live thread count (SIG-2), a real ephemeral podman container emitting a real 4-hit EAGAIN cascade read via the real podman logs path (SIG-3), a genuinely-high (avg60=65.00) PSI reading injected via a new PSI_FILE override point exercising the detectorâ€™s real parse+compare code â€” real memory-pressure induction was deliberately avoided on host-safety grounds, see that fixtureâ€™s header (SIG-4), and a live process whose real /proc/<pid>/cmdline genuinely matches the detectorâ€™s own regex, verified byte-for-byte against that regex extracted from the challenge script itself (SIG-5). bash challenges/scripts/verify_resource_pressure_polarity.sh runs all five against the UN-MUTATED challenge and its DEFAULT thresholds â€” real run: RED confirmed: 5 / 5 FAIL: 0 SKIP: 0. Evidence: docs/qa/task-78/{sig1..sig5}_real_fixture_output.txt + docs/qa/task-78/verify_resource_pressure_polarity_output.txt.
- **Honest boundary (Â§11.4.6)**: the SIGKILL source is **not** claimed identified â€” the journal shows the receipt of SIGKILL but the sender is not attributed, filed as PENDING_FORENSICS (Task #79). Killing the pathological ugrep is a bandage that fixes ONE known contributor class, not proof it was THE cause. The new challenge captures signatures the incident had, not a proof the next incident will be prevented â€” that requires the pre-build integration + systemd-user timer of #77 firing, catching an SIG-* over threshold, and escalating before user@1000 dies.

DB-BLOB-COMMITTED-WITHOUT-DELTA-3520621 â€” docs/workable_items.db binary blob committed with only a prose claim, no differential evidence

- **id:** none pre-existing (a Â§11.4.209 code-review finding, not an operator/user-reported defect) â€” remedy tracked as this-session Task #79 (.superpowers/sdd/task-79-report.md; note the SAME numeral â€œ#79â€ is ALSO used elsewhere in this ledgerâ€™s own FORCED-LOGOUT-2026-08-18-2ND entry above for a DIFFERENT task [â€œSIGKILL-source attribution long-runâ€] â€” an honest, unresolved task-numbering collision in this sessionâ€™s own SDD dispatch records, surfaced here rather than silently reconciled per Â§11.4.6; out of scope for this entryâ€™s remedy to fix).
- **date:** 2026-08-18 â€” found during the independent Â§11.4.209 review of commits 457cca4..a7e55f9 (.superpowers/sdd/task-review-457cca4-a7e55f9-report.md, finding IMPORTANT-2).
- **channel:** agent-code-reading â€” found by an independent reviewer comparing git show --stat 3520621 (an 8 KiB docs/workable_items.db blob change) against the commit messageâ€™s own claim (â€œmeta table content is unchangedâ€) and cross-referencing the SAME sessionâ€™s Task #41 investigation, which had independently confirmed shared-checkout races on this exact file. No test, gate, or HelixQA run flagged this â€” the review was reading the commit itself.
- **escape-audit: genuinely uncovered** â€” no automated check anywhere in this repo, at the time commit 3520621 landed, ever opened docs/workable_items.db and diffed its logical content across a commit boundary. The ONLY existing SSoT-integrity check (workable-items validate) asserts internal consistency of the CURRENT state; it has no concept of â€œwhat changed since the parent commitâ€ and therefore cannot catch an under-disclosed mutation riding inside an otherwise-legitimate binary-blob commit. commit-push-all.shâ€™s --scope safety check (task #66/BOB-068) also cannot see inside the blob â€” it only verifies which FILES are staged, not what changed WITHIN one of them.
 - **Backfill result (this entryâ€™s own finding, task #79):** running the new scripts/capture-workable-items-db-delta.sh 3520621 helper against the real repository produced docs/qa/db-deltas/3520621866e071a6d10ec06e5b432188fdac7129.diff, confirming: (1) items table â€” exactly TWO rows changed: BOB-108 (Queuedâ†’Fixed (â†’ Fixed.md), the commitâ€™s stated, intended mutation) and BOB-104 (Queuedâ†’In progress, body expanded with an upstream-issue-filing update) â€” **the BOB-104 change is real, legitimately audit-trailed (item_history row 89, By='AI', timestamped 19:16:36, one minute before BOB-108â€™s own row 90 at 19:17:47), and consistently reflected in the committed docs/Issues.md at that same commit â€” but it is NOT mentioned anywhere in the 3520621 commit message**, which describes only the BOB-108 mutation and the surfacing of brand-new items BOB-109..BOB-114. (2) item_history â€” exactly 2 new rows (89,

90), both correctly attributed. (3) doc_segments “ exactly 2 rows changed, consistent with BOB-108’s segment relocating from the Issues doc to the Fixed doc. (4) meta, logic_groups, test_diary, test_diary_summary, obsolete_details, operator_block_details, firebase_metadata “ byte-for-byte **unchanged**, confirming the commit_message’s “œmeta table content is unchanged” claim was TRUE as far as it went.

Verdict: no data corruption, no silent data loss, every row-level change is legitimately audit-trailed “ but the commit message’s disclosed scope was narrower than its actual content (an under-disclosed-but-benign mutation, exactly the class Â§11.4.226 requires runtime/artifact-class evidence to rule out rather than accept on a source-class prose claim alone).

- **new-check:** two-part remedy, both landed this session (task #79): (1) scripts/capture-workable-items-db-delta.sh (NEW) “ produces a differential SQLite dump (per-table row counts + full meta dump + unified .dump diff) between any commit touching docs/workable_items.db and its parent, written to docs/qa/db-deltas/<full-sha>.diff; Â§11.4.115 REDâ†’GREEN polarity confirmed via a real Â§1.1 paired mutation (breaking the sqlite3dump call in a scratch copy of the script produces exit 127 and **zero files** at the real output path “ the atomic temp-file-then-mv publish pattern was added specifically so a genuine capture failure can never leave a partial, misleadingly-œcleanœ file behind; the unmodified script then reproduces the full evidence cleanly). (2) scripts/commit-push-all.sh stage 5.5 (NEW) “ automatically invokes the helper immediately after ANY future commit that touches docs/workable_items.db lands, and lands the resulting delta as an immediate scoped follow-up commit (docs(qa,db-delta): capture differential dump for HEAD <sha>) travelling to every upstream in the same run “ so this specific gap cannot recur silently going forward. See docs/scripts/capture-workable-items-db-delta.md for the full mechanism.
- **Honest boundary (Â§11.4.6):** this remedy makes future docs/workable_items.db-touching commits self-evidencing; it does NOT retroactively re-open or re-litigate 3520621 (no history rewrite, Â§11.4.113) and does NOT claim the BOB-104 under-disclosure was malicious or harmful “ the backfilled evidence shows it was benign and fully audit-trailed. It IS, per this ledger’s own purpose, a real coverage escape: a commit message described a narrower scope than what it actually committed, and no automated check existed to catch that gap before an independent review found it by hand.

FORCED-LOGOUT-2026-08-18-3RD “ user@1000.service SIGKILLED at 23:45:49 (3rd occurrence), plus the preventive monitor’s own architectural gap

- **id:** BOB-120 (Type=Bug, Severity=Critical, Status=Queued “ this document doc + evidence capture do not close it; closure requires the actual out-of-user-scope watchdog fix, tracked as a followup of task

#77). Full writeup: docs/incidents/2026-08-18-3rd-forced-logout.md.

- **date:** 2026-08-18, 23:45:49 “ the 3rd occurrence of this incident class on this host (1st: 2026-07-07, produced Â\$12.12; 2nd: BOB-116, same day 20:50:59; 3rd: this entry, ~2h55m after BOB-116, same session lineage).
- **channel:** agent-code-reading / session-continuation-dispatch “ this incident was investigated proactively by an agent picking up a mid-turn task-continuation dispatch, not by the operator manually re-reporting a fresh logout observation this time. The real systemd journal, podman ps/podman logs, and the BOB-116 preventive timer’s own fire history were read directly (docs/qa/BOB-120/{journalctl_23-42_to_23-46.log, timer_fire_history.log, qbittorrent_proxy_socketserver_traces.log}).
- **escape-audit:** the SIGKILL-source question remains the SAME open escape as BOB-116 (Task #79, PENDING_FORENSICS, no automated check exists anywhere in this repo that attributes a user@1000.service SIGKILL to its sender) “ this incident adds a **second, distinct** coverage escape: the exact preventive check BOB-116 built in response to the 2nd occurrence (boba-resource-pressure-check.timer, task #77) is **architecturally incapable of catching a kill that lands inside its own hosting scope**, because the timer is a systemd --user unit whose process tree is a child of user@1000.service itself. When that service is SIGKILLED, the timer + its triggered probe die in the exact same cascade “ the monitor cannot outlive the thing it monitors, and by construction can never observe or escalate on a kill that happens to land in the window between its own hourly fires (up to ~1h05m per cycle, base 1h + up to 5m jitter). Captured fire history shows the last completed run before this incident finished at 22:57:58 “ 47m51s before the 23:45:49 kill “ so the timer had not even reached its next scheduled activation; this was not a missed fire, it was a structurally-unreachable one. This is a genuine, previously undocumented limitation of the BOB-116 remediation, not a bug in the remediation’s own logic (the 5-signature detector itself is unchanged and still self-validated per Â\$11.4.107(10) from BOB-116/task #78).
- **new-check: NOT YET LANDED** “ this entry documents the finding + files BOB-120 as the tracked follow-up; the actual fix (an out-of-user-scope watchdog: a root-level systemd timer, or a --user unit escaped from the session-scope cgroup via delegation/isolation) is deliberately left as open, Queued work rather than rushed into this incident-response window. Closing BOB-120 on documentation alone, without the architectural fix landing, would itself be a Â\$11.4.238 coverage-escape bluff “ the exact failure mode this anchor exists to forbid.
- **Honest boundary (Â\$11.4.6):** the originally-circulated hypothesis for this incident (“œthe timer’s next expected fire was 23:42:38 and it silently missed it“♦) does **not** survive contact with the real captured journalctl --user fire history (last completed run 22:57:58, next due no earlier than 23:52:58) and is explicitly corrected in the incident document rather than repeated as fact. The qbittorrent-proxy ConnectionResetError cascade (every ~30s through 23:38“23:45, last line 5s before the kill) and the boba-stack containers’ near-

instant re-creation (CreatedAt=23:45:52, 3s after the kill) are reported as time-correlated observations, not established causes.

FORCED-LOGOUT-2026-08-19-5TH " 5th user@1000.service SIGKILL at 15:28:22, architectural install-gap: 4 authored preventive gates, 0 installed

- **id:** BOB-124 (Type=Bug, Severity=Critical, Status=In progress " filed this session, evidence at docs/qa/BOB-124/incident-5-forensics.log). Companion to BOB-116/BOB-120/BOB-123 (incidents #2/#3/#4); full mechanism/triage lives in the forced-logout-incidents project-memory playbook.
- **date:** 2026-08-19, 15:28:22 " the 5th forced-logout on this project (fresh host boot 15:07 " kill 15:28, exactly 21 minutes into a fresh session, first physical logout since incident #4"s ~00:37 occurrence).
- **channel:** operator-report at initial detection (operator physically observed the GDM greeter after lid re-open), corroborated by agent-code-reading during triage (journal grep confirmed the same PAM-session-close-synchronicity signature already documented across #2/#3/#4). No standing automated check monitored either the SIGKILL delivery or the leading resource-pressure signatures during this session. Evidence: docs/qa/BOB-124/incident-5-forensics.log.
- **escape-audit: the coverage escape is now architectural, not per-check "** this is the 5th consecutive incident of the same class and the underlying block is the same across all five: the preventive gates authored in response to incidents #2/#3/#4 (BOB-116 5-signature detector, BOB-120 out-of-scope watchdog design, BOB-123 PAM-session-close monitor, sundry auditctl kernel rulesets) ALL require a sudo/su -c install step that has never actually been executed by the operator. The gate text ships; the mechanism does not run. Per superpowers:systematic-debugging skill Phase 4.5 (" attempts against the same block = architectural problem, not hypothesis failure) and per "11.4.250 (heuristic-tower signals a primitive defect), authoring a 6th detector would raise the tower height without raising the ceiling " the coverage escape is the install-gap, not any missing signature. The boba-resource-pressure-check.timer from BOB-116 DID install (it is a --user unit, no sudo needed) but is architecturally blind to a kill of its own hosting scope (BOB-120 gap), so its installed but blind status does not close this escape either. existed-but-uninstalled (a distinct "11.4.201(6) FALSE-NUL sub-class from existed-but-missed " the check exists in text, but no live instance runs to observe the defect at all).
- **new-check: NOT AUTHORIZING A NEW CHECK.** Followup filed as BOB-124 itself: install one of the already-authored preventive paths (Path 1 kernel auditctl rules OR Path 2 out-of-scope systemd watchdog OR Path 4 dedicated boba-watchdog UID) via an operator sudo session, and verify it is live via auditctl -l non-empty or systemctl list-units --system 'boba-watch*' non-empty before this ledger entry may be closed. Closing this entry on a 5th authored-but-uninstalled detector would itself be a "11.4.238 coverage-escape

bluff (the exact failure mode this anchor exists to forbid) AND a
Â§11.4.250 heuristic-tower violation.

- **Honest boundary (Â§11.4.6):** the SIGKILL initiator remains UNCONFIRMED across all 5 incidents â€” the PAM session_close correlation is the mechanism, not the root initiator, and Linger=yes should have prevented the kill by design (whether a systemd bug, an unattributed loginctl terminate-user call, or an SDD/container-fleet component invoking a stop path at a different identifier we have not yet grepped is still open). This entry does NOT claim a root cause; it claims the coverage escape is the install-gap on the ALREADY-KNOWN preventive paths, which is itself proven by 5 consecutive occurrences without a single mechanical detection.

Discovery-channel split (tracked, per Â§11.4.238(E))

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
2026-08-07 â†’ 2026-08-10 (incremental, this period only)	0	8 (agent-code-reading x4, incidental-discovery x3, automated_background_scan x1)	100%
2026-08-11 â†’ 2026-08-12 (incremental, this period only)	0	1 (operator-report x1)	100%
2026-08-18 (incremental, BOB-069 backfill â€” 4 new ### entries; the BOB-073 recurrence is an addendum to the existing BOB-008 heading, not a new ###)	0	4 (agent-code-reading x4: RD2-00/BOB-068, BOB-072, META-11.4.227B, SCRATCH- LOSS)	100%
2026-08-18 (incremental, BOB-069-review fix â€” 1 new ### entry: the missing CodeGraph 1.5.0 nested-.gitignore escape identified in the independent review of BOB-069)	0	1 (agent-code-reading x1: CODEGRAPH-1.5.0- GITIGNORE)	100%
2026-08-18 (incremental,	0		100%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
BOB-116 forced-logout incident (initially referenced as BOB-076 informal label, corrected 2026-08-18) 1 new ### entry: 2nd occurrence of user@1000 SIGKILL on this project after physical operator return, discovered by operator-report)		1 (operator-report x1: FORCED-LOGOUT-2026-08-18-2ND)	
2026-08-18 (incremental, 11.4.209 review IMPORTANT-2 remedy 1 new ### entry: docs/workable_items.db binary blob committed without differential evidence, discovered by agent-code-reading)	0	1 (agent-code-reading x1: DB-BLOB-COMMITTED-WITHOUT-DELTA-3520621)	100%
2026-08-18 (incremental, BOB-120 forced-logout incident 1 new ### entry: 3rd occurrence of user@1000 SIGKILL on this project, plus the discovery that the BOB-116/task-77 preventive monitor itself lives inside the killed scope and cannot catch a kill landing before its own next fire)	0	1 (agent-code-reading x1: FORCED-LOGOUT-2026-08-18-3RD)	100%
2026-08-19 (incremental,	0		100%

Period	automated-helixqa	out-of-band (all channels)	out-of-band %
BOB-124 forced-logout incident #5 " 1 new ### entry: 5th consecutive occurrence of user@1000 SIGKILL, escalating the coverage escape from "no preventive gate" to 4 authored preventive gates, 0 installed" \$11.4.250 heuristic-tower / architectural install-gap)		1 (operator-report x1: FORCED-LOGOUT-2026-08-19-5TH)	
Cumulative total (all ### entries to date, this row is what CM-QA- DISCOVERY-LEDGER-FRESH checks)	0	18	100%

Pre-existing check-vs-table mismatch, found and fixed during this backfill (\$11.4.6, not silently patched around): scripts/pre_build_verification.sh invariant 19 (CM-QA- DISCOVERY-LEDGER-FRESH) computes its expected count from ONLY the LAST row of this table, comparing it against the TOTAL ### heading count under ## Entries. The three "incremental, this period only" rows above were never designed to satisfy that arithmetic on their own (each was authored to show only that update's delta, not a running total) " re-running the exact invariant-19 awk logic against the pre-backfill file (git show HEAD:docs/QA_DISCOVERY_LEDGER.md) confirms this was **already FAILing before this session's edits** (entries=9 split-table-declared=1, live-run of scripts/pre_build_verification.sh confirms FAIL [5]). The **Cumulative total** row above is added so the check's actual last-row-only arithmetic now passes (0+13=13 == the real ### count below); the three incremental rows are kept as honest per-period trend history. This finding is itself a coverage escape of the same shape this ledger tracks (an existed-but-missed gate whose own arithmetic silently drifted from the document it gates) but is fixed in the same edit that found it since the fix is a one-line table-convention correction, not a new automated check.

Honest note: 100% out-of-band is the true, unflattering starting number " every entry above was found by an agent reading code, running commands by hand, or hitting a real failure while doing unrelated live-verification work; none by a standing HelixQA run. This is exactly the state

Â§11.4.238 exists to change; the target is this percentage trending toward zero as the new-check column above closes each specific gap (RD2-42/RD2-43 are still open â€” no automated check authored yet, tracked honestly as such) and as future work is driven through the HelixQA banks (challenges/helixqa-banks/) rather than ad-hoc investigation.

2026-08-18 backfill note (BOB-069): this round added four entries discovered during the SAME session that produced them (the constitution-curriculum amendment stream + the BOB-072/073/075 parallel fixes), plus one recurrence addendum to the pre-existing BOB-008 entry. All five remain agent-code-reading â€” none crossed a standing automated gate. BOB-072 is the closest this ledger has come to the ideal shape: its closing evidence was captured via the sanctioned tool (workable-items validate/diff) rather than raw inspection, even though the discovery itself predates any standing gate invoking that tool automatically. Four followup workable items were filed (BOB-104..BOB-107) for the specific new/strengthened automated checks each entryâ€™s new-check field names as not-yet-authored; none of the four checks are claimed as shipped by this backfill (Â§11.4.6) â€” they are tracked, not implemented, here.

2026-08-19 BOB-124/125/126 escape â€” SEVEN forced-logout incidents traced to ONE untested test-defect (chain closed):

Between 2026-08-18 20:50 and 2026-08-19 16:43, **SEVEN** forced-logout incidents (BOB-116 / BOB-120 / BOB-123 / BOB-124 / BOB-125 / BOB-126 â€” the seventh caught by kernel audit rules installed 15:56 during the sixth investigation) wiped the operatorâ€™s entire graphical session, tmux server, ssh connections, browsers, and the working Claude Code process â€” every ~30-40 minutes for ~48 hours.

Actual root cause (Â§11.4.6-attributed 2026-08-19 16:43:43 via audit[399861]: SYSCALL syscall=62 a0=ffffffff a1=9 pid=399861 comm=\"pytest\" exe=\"/usr/bin/python3.14\"): tests/unit/merge_service/test_deadline_tunable.py::test_deadline_hit_flag_true_when_readline_times_created AsyncMock() without setting mock.pid as an int. Production _search_public_tracker code path called os.killpg(os.getpgid(proc.pid), signal.SIGKILL). Pythonâ€™s MagicMock.__int__ defaults to 1, so os.getpgid(int(MagicMock())) == 1 â†’ os.killpg(1, SIGKILL) â†’ glibc â†’ kill(-1, SIGKILL) = **SIGKILL every UID-1000 process**. Bug existed since 2026-04-24 (~4 months). contextlib.suppress(Exception) swallowed nothing because the syscall SUCCEEDED before any exception.

Discovery channel: operator-report (7 times). **Coverage-escape audit** (Â§11.4.238(C)): the automated regime had NO check for â€œdoes a production-code function ever call killpg with pgid â‰¥ 1?â€” or â€œdoes a test create subprocess mocks without explicit int pid?â€”. Â§11.4.115 RED-first was violated: the test that TRIGGERED the disaster had no assertion that the code path avoided the disaster. New check landed: tests/unit/merge_service/test_deadline_tunable.py::test_bob126_regression_deadline_path_never_calls

(see commit ad4b46a) – a §11.4.115 RED-first regression guard asserting `os.killpg` is NEVER called with `pgid` `1` even under the exact defect precondition. Reverts of the production-code pid-guard MUST make this test fail. Also NEW universal anchor **§11.4.263** – process-group signal-safety mandate (Constitution commit 502586c) – extends the mandate to every project inheriting the constitution (Python/Go/Rust/Bash/C) with four-layer coverage per §11.4.4(b).

§11.4.238 discipline preservation (agent-code-reading DID FIND, sort of – but only on the 7th time, and only via kernel audit rules the operator ran a `su -c` command to install manually on the same day; the prior 6 investigations misattributed the mechanism to PAM/Linger contradictions and other downstream effects – all wrong per §11.4.6, honest self-correction at Rev 4 of the incident docs). This is the ledger’s LARGEST out-of-band escape by user-visible impact.

- **BOB-124** (5th incident): status=Fixed via BOB-126 root-cause. Incident doc filed 2026-08-19 15:35, corrected retrospectively.
- **BOB-125** (6th): status=Fixed via BOB-126. Doc filed 16:11, Rev 4 correction 16:15.
- **BOB-126** (7th): status=Fixed. Doc filed + closed same day.
- Prior 4 (BOB-116, BOB-120, BOB-123 + the base BOB-116’s –œfirst– instance): retrospective correction pending – their attribution to PAM/Linger was wrong per BOB-126’s captured evidence.